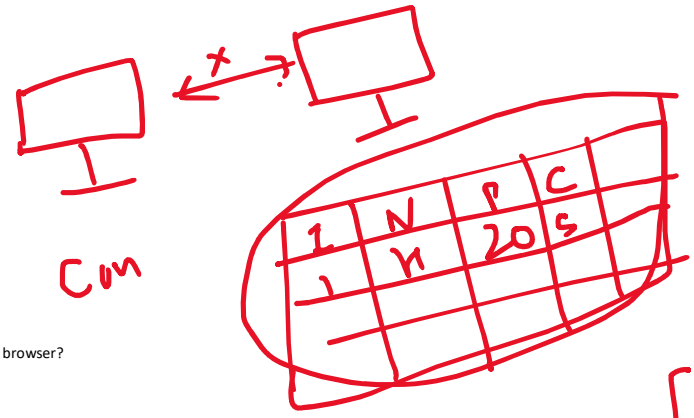
Client Architecture, APIs and Spring Introduction

Tuesday, October 6, 2026 12:02 AM



1. From Listenify to a browser

Does printing a song in IntelliJ send it to another laptop?

```
package lecture5.console;

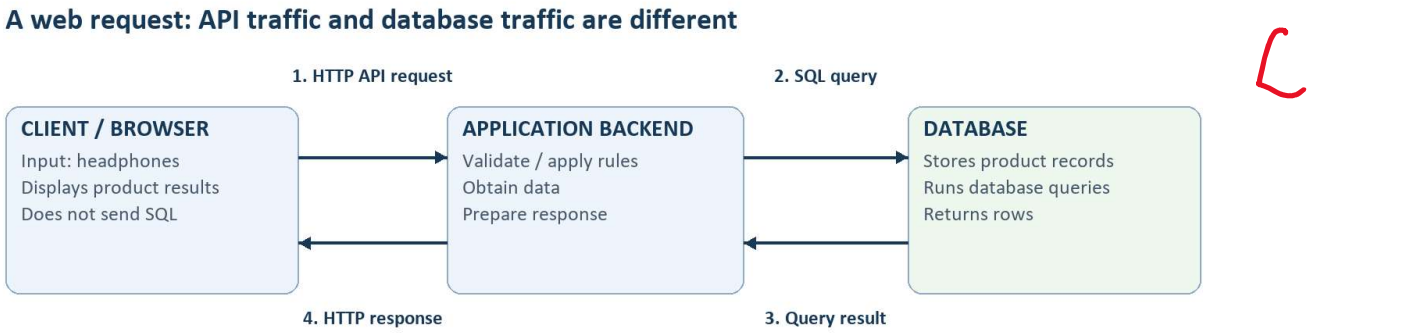
public class ConsoleLookupDemo {
    static String findSong(String title) {
        if ("Sunrise".equals(title)) return "Sunrise (3.5 min)";
        return "Song not found";
    }

    public static void main(String[] args) {
        System.out.println(findSong("Sunrise"));
        System.out.println(findSong("Missing"));
    }
}
```

Our songs are printed in IntelliJ. How would a user on another machine request them from a browser?

Does a Java method returning String automatically expose a web API?

2. Searching for headphones



Simplified shopping example. A backend may use memory/cache instead of a database.

If the website can display a button, does that prove the backend action is complete?

3. Read a request: URL, host, port and path

```
findSong("Sunrise")
```

```
GET  http://localhost:8081/songs?title=Sunrise
```

Q- What does each part of this address identify?

```
http     |     localhost     |     8081     |     /songs     |     title=Sunrise
```

Q - When you open localhost on your laptop, which computer are you contacting?

Q - Can one computer run multiple services? How would their addresses differ?

Q - Where does the extra search value appear in the URL?

Google search — inspect the request URL in the browser Network panel.

4. API request versus database query

Search request: API call and database query

Illustrative web Listenify search. A console method call is not a network API request.



Two different operations

API call: client asks the backend. Database query: backend asks the database.
The backend may also use memory, a cache or a search index instead of a database.

Example request

GET /songs?title=Sunrise -> 200 OK with JSON song data

```
GET /songs?title=Sunrise
SELECT title, duration_minutes FROM songs WHERE title = ?;
```

Q- Which request is sent by the client, and which operation could happen inside the backend?

Q - Does the client need to know our database tables or how the data is stored?

Q - Who supplies the value represented by ? in the SQL example?

Google search — locate the response status in the Network panel.

Q - Can GET /server-time work without a database?

Q - Does every successful API response imply a database was updated?

Q - Does our browser send the SQL query or database operation we need to perform?

5. HTTP anatomy, methods and status codes

HTTP MESSAGE ANATOMY: GET /songs

REQUEST

```
GET /songs HTTP/1.1
METHOD | PATH | VERSION
Host: localhost:8081
HEADERS
Accept: application/json
[blank line]
[no body in this example]
```

RESPONSE

```
HTTP/1.1 200 OK
VERSION | STATUS CODE | REASON PHRASE
Content-Type: application/json
HEADER
[blank line]
[{"title":"Sunrise",
BODY
  "durationMinutes":3.5}]
```

What travels to the server? What comes back to the client?

Identify the method, path, headers, response status and body in these messages.

Which header describes the format being sent? Which describes the format requested?

CHOOSE THE METHOD / PREDICT THE OUTCOME

METHODS: GET | POST | PUT | DELETE

Retrieve songs: ? Create/submit: ?

Replace song details: ? Delete a song: ?

STATUS CODES: 200 | 201 | 400 | 404 | 405 | 500

Request succeeds: ? Resource created: ?

Invalid input: ? Unknown resource/path: ?

Method not allowed: ? Unexpected server failure: ?

Can a request reach the server and still fail?

Can a request reach the server and still fail?

DESIGN SKETCH: ADD A SONG TO PLAYLIST 1

REQUEST

```
POST /playlists/1/songs HTTP/1.1
```
METHOD | PATH | VERSION

```
Host: localhost:8080
```
HEADERS

```
Content-Type: application/json
Accept: application/json
[blank line]
{"songTitle":"Sunrise"}
```
BODY

RESPONSE

```
HTTP/1.1 201 Created
```
STATUS LINE

```
Content-Type: application/json
```
HEADER

```
[blank line]
{"entryId":7,
 "songTitle":"Sunrise"}
```
BODY

What travels to the server? What comes back to the client?

Design an endpoint that adds a song to playlist 1. Where does the song title go?

What changed between the GET and POST examples?

Should GET trigger a purchase or delete a song?

Does naming a request POST automatically save anything?

6. Safe and idempotent: predict repeated requests

REPEATING A REQUEST: WHAT HAPPENS TO THE CART?

Initial Diet Coke quantity: 0

Request A: set quantity to 3 → first: ? → repeat: ?

Request B: add 3 more cans → first: ? → repeat: ?

Which requests are safe? Which are idempotent?

Is GET /deletePatient?id=7 a good contract?

Does idempotent mean the same status every time?

Can clicking Submit twice matter?

7. JSON and the UI/backend boundary

DATA AND DISPLAY: TRACE THE VALUES

Java response data → JSON text → JavaScript values → HTML element

JSON: {"title":"Sunrise","durationMinutes":3.5}

JavaScript: song.title and song.durationMinutes

What does the browser receive? Who decides how it looks?

```
{"title":"Sunrise","durationMinutes":3.5}

[
  {"title":"Sunrise","durationMinutes":3.5},
  {"title":"Road Trip","durationMinutes":4.0}
]

const song = { title: "Sunrise", durationMinutes: 3.5 };

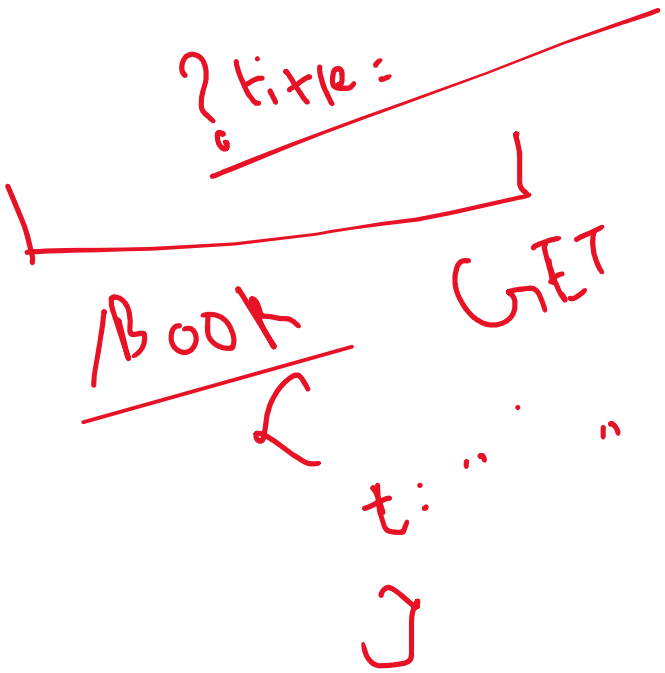
const heading = document.getElementById("header1");

heading.textContent = `${song.title} (${song.durationMinutes} min)`;
```

Does the browser receive our Java Song instance?

Who turns the response data into a card or table?

Is seeing JSON in a browser a completed frontend?



8. Design a bookstore search request

A user enters Java in a bookstore search box and wants to see matching books.

What HTTP method and path would you choose? Where should the search text go?

Sketch one successful JSON response and choose its status.

If no books match, must that be an error? Is this operation safe and idempotent?

```
METHOD + PATH: _____   /books
INPUT: _____  ?title=bookTitle
SUCCESS STATUS + JSON: _____  200 {title}
NO MATCHES: _____ 404   - 200 []
SAFE? _____   IDEMPOTENT? _____
```

9. Run one endpoint in Java

```
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;

public class HttpDemo {
    public static void main(String[] args) throws java.io.IOException {

        HttpServer server = HttpServer.create(
                new InetSocketAddress("127.0.0.1", 8081), 0);

        server.createContext("/songs", exchange -> {
            String json = "[{\"title\":\"Sunrise\",\"durationMinutes\":3.5}]";
            byte[] bytes = json.getBytes(StandardCharsets.UTF_8);
            exchange.getResponseHeaders().set("Content-
Type", "application/json");

            exchange.sendResponseHeaders(200, bytes.length);

            try (var output = exchange.getResponseBody()) {
                output.write(bytes);
            }
        });


        server.start();
        System.out.println("Open  http://127.0.0.1:8081/songs");
    }
}
```

Where is the server listening? Which path has a handler?

Where are the response status, format and body set?

What keeps the application running after main finishes?

10. Browser and Postman: inspect and predict

ONE JAVA SERVER: PREDICT EACH REQUEST

Browser / Postman → http://127.0.0.1:8081 → Java HttpDemo

GET /songs → status: ? body: ?

POST /songs → status: ? body: ?

GET /missing → status: ? | server stopped → ?

Browser: open http://127.0.0.1:8081/songs and inspect Network.

Postman: use the same address. Select GET, send, then select POST and predict before sending.

Call http://127.0.0.1:8081/missing. Then stop the Java application and retry /songs.

```
Record for each request:
URL: _________
Method: _________
Response status: _________
Content-Type: _________
Response body: _________
```

Did choosing POST itself change what our handler does?

How is a missing endpoint different from a server that is not running?

10. Extension: allow only GET

Before adding this check: predict POST /songs. After adding it: predict the status, headers and body. Where should the check go?

```
if (!exchange.getRequestMethod().equals("GET")) {
    exchange.getResponseHeaders().set("Allow", "GET");
    exchange.sendResponseHeaders(405, -1);
    exchange.close();
    return;
}
```

Compare GET /songs, POST /songs and GET /missing after the change. Does the method name alone enforce our contract?

11. Stateless HTTP: requests, users and stored playlists

TRACE THREE INDEPENDENT REQUESTS

Request 1: GET /playlists/7 → running application

Request 2: GET /playlists/8 → same running application

Asha: GET /my-playlists | Rahul: GET /my-playlists

Same path: what information could identify the user?

```
GET /my-playlists HTTP/1.1
Host: localhost:8080
Cookie: sessionId=abc123
```

Does the second playlist request need the result of the first?

Does stateless HTTP mean every request starts a new Java server?

Can the application store a playlist and retrieve it on a later request?

Does storing playlists in a database violate HTTP statelessness?

Do our current Listenify albums survive an application restart? Why?

12. One-tier, two-tier and three-tier architecture

DRAW THE BOUNDARIES BEFORE COUNTING TIERS

One tier: [Desktop UI + application logic + local data]

Two tiers: [Desktop client / application logic] ↔ [Database server]

Three tiers: [Browser UI] ↔ [Application server] ↔ [Database]

Place a local Paint application, MySQL Workbench connected to a remote database, and a web application in these sketches.

In each sketch, where does the user interact? Where is application logic? Where is data?

If we create three Java packages on one laptop, have we deployed three tiers?

13. Spring Boot: generate, inspect and run

BUILD TIME AND RUN TIME

Java source + pom.xml → Maven → compiled / packaged application

JVM process: [application + Spring + embedded Tomcat]

Browser request → Tomcat → Spring MVC → application handler

Assign a role: Spring | Spring Boot | Maven | Tomcat | JVM | IntelliJ

Which parts did we write manually in HttpDemo? Which work should a framework help with?

What does Spring provide? What does Spring Boot simplify? Who builds the project? Who executes Java? Who accepts HTTP traffic?

https://start.spring.io

```
Project: Maven | Language: Java | Spring Boot: 4.1.1
Group: com.accio | Artifact: lecture5-spring-demo
Name: Lecture5Application | Package: com.accio.lecture5
Packaging: Jar | Java: 25 | Dependency: Spring Web
```

Generate → extract → open the folder containing pom.xml in IntelliJ → check JDK → let Maven import finish. Locate pom.xml and main. What did the generator create? Did it implement /hello?

```
package com.accio.lecture5;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Lecture5Application {
    public static void main(String[] args) {
        SpringApplication.run(Lecture5Application.class, args);
    }
}
spring.application.name=lecture5-spring-demo
server.address=127.0.0.1
server.port=8080
```

Run the generated main class. Check the startup log, then request http://127.0.0.1:8080/hello.

Predict the result before sending. How would you distinguish a missing route from a stopped server?

Did we install Tomcat separately?

Does Maven answer browser requests?

Whose computer does localhost identify?

15. Maven and pom.xml: dependencies and packaging

TRACE A DEPENDENCY AND A BUILD

pom.xml dependency: groupId + artifactId + version (GAV)

Maven lookup: local .m2 repository → configured remote repositories if needed

Project source + resources + tests → build → target/

Inspect: IntelliJ External Libraries | .m2 | target/

```
lecture5-spring-demo/
  pom.xml
  mvnw.cmd
  src/main/java/                [ locate application code ]
  src/main/resources/           [ locate application.properties ]
  src/test/java/                [ locate generated test ]
  target/                      [ inspect after building ]
```

Open the actual pom.xml. Find the parent, project GAV, Java version, dependencies, test scope and Boot plugin.

How does Maven identify one library?

Why do we see libraries we did not list directly?

Which dependencies belong to tests?

Where would we add a new application dependency?

Inspect the resolved libraries and C:\Users\Akash\.m2\repository. Where is the built project JAR?

```
.\mvnw.cmd package
```

```
java -jar target/lecture5-spring-demo-0.0.1-SNAPSHOT.jar
```

Stop the IDE application before running its executable JAR. Check the startup log and request /hello again.

Can packaging succeed while /hello still returns 404? Does a successful build prove an endpoint exists?

If the IDE application and the JAR both try to use port 8080, what would you predict?